



IQPREC

Intelligence. Precision. Every Gameweek.

Complete Claude Code Build Bible

Stack: HTML + CSS + Vanilla JS + Node.js

No React. No Build Step. Works on Any Machine.

Jacob Qumsiyeh & Simon Haddad · Co-Founders · iqprec.com

This is the complete IQPREC build bible — rebuilt from scratch using HTML, CSS, and Vanilla JavaScript. No React. No Vite. No build step. No node_modules nightmare. You open a file and it works. Every single feature from the original plan is here — just built the way the web was meant to work.

Why This Stack Is Better For You Right Now

Setup time	1+ hour npm installs	5 minutes — no install needed
To see changes	Wait for Vite to compile	Press F5 in browser — instant
On old laptop	Slow, laggy, frustrating	Runs perfectly on any machine
node_modules size	500+ packages, 200MB+	Server only, 50 packages, 30MB
Debugging	React DevTools, build errors	F12 browser DevTools, simple
Deployment	Build step required	Copy files and done
End result for user	Identical — user never knows	Identical — user never knows

IMPORTANT: The product IQPREC looks and works identically for the user. An Arab FPL manager visiting iqprec.com sees the same dark navy design, the same Arabic AI recommendations, the same competitions, the same everything. The only difference is YOU as the developer no longer spend hours fighting npm. You write a file, refresh the browser, and see it. That is it.

30-Day Build Overview

1	Project Structure + Design System	Foundation	Folder setup, CSS variables, IQPREC design tokens, fonts
2	HTML Templates + Components	Foundation	Base layout, sidebar, nav, reusable HTML patterns
3	Landing Page	Foundation	Hero, features, pricing, competitions, FAQ, footer
4	Pentagon Security + Database	Security	All 20 Supabase tables, RLS policies, security middleware
5	Auth Backend	Security	Register, login, JWT RS256, refresh, trial system
6	Auth Frontend + Trial	Security	Login/register HTML pages, trial countdown, referral system
7	FPL Data Engine	Data	FPL API, Redis cache, active players only, Arab players sync
8	Billing + Receipts	Data	Stripe \$15/\$110, receipt emails, trial expiry, Paytabs v1.1
9	Dashboard Page	Data	Stats, squad view, captain card, competitions widget
10	AI Expert Engine	AI Core	Claude API, expert FPL rules, community data injection
11	Lineup + Captain Pages	AI	Pitch SVG view, confidence display, Arabic reasoning
12	Transfer + Differentials	AI	Hit analysis, hidden gems, community ownership context
13	Player Intel + Arab Stars	AI	Scouting reports, Arab player tracker, active only
14	Chip Planner + Mini-League	AI	Season timeline, DGW markers, rival tracking
15	AI Chat + Voice Summary	AI	Streaming SSE chat, Arabic voice briefing script
16	What If + Feedback Loop	AI	Season comparison, H2H tracker, gameweek results
17	Community + Share Cards	Community	Leaderboard, Canvas API WhatsApp 1080x1080 cards
18	Competitions System	Community	Milestone tracker, referral verification, prize management
19	Ask Me + Notifications	Community	Human support inbox, 9 email templates, Telegram bot
20	Onboarding Tutorial	Community	6-step guided tour using vanilla JS highlight overlay
21	Admin Panel + Analytics	Ops	Stats dashboard, competition management, PostHog
22	Legal Pages	Polish	About Us, Privacy Policy, Terms, Cookie Policy
23	Full Landing Page Polish	Polish	Hero animation, competitions preview, OG meta tags
24	Security Audit	Polish	All 8 layers tested, active player check, SSL A+
25	Performance	Polish	Page speed, lazy loading, cache headers, minification
26	Mobile Polish	Polish	375px every page, touch targets, PWA manifest
27	RTL + Accessibility	Polish	Full Arabic RTL audit, WCAG 2.1 AA, aria labels

28	Production Deployment	Launch	Contabo Nginx, PM2, DNS, certbot SSL — no build step needed
29	Smoke Test + Fixes	Launch	Every flow end-to-end, zero P0 issues confirmed
30	Launch Day	Launch	WhatsApp 200 contacts, Instagram, Competition 1 opens



Master CLAUDE.md System Prompt

This is already in your CLAUDE.md file. Shown here for reference.

PROJECT IDENTITY

- Product: IQPREC | Domain: iqprec.com | IQ = Intelligence, PREC = Precision
- Tagline EN: Intelligence. Precision. Every Gameweek.
- Co-Founders: Jacob Qumsiyeh (Developer) and Simon Haddad (Finance and Gaming)
- Stack: HTML5 + CSS3 + Vanilla JS (frontend) / Node.js + Express (backend) / Supabase / Redis / Claude AI / Stripe
- NO REACT. NO VITE. NO BUILD STEP. Pure HTML files served by Express or Nginx directly.
- Daily priority: UI/HTML structure first, CSS styling second, JS functionality third, Backend fourth, Security fifth.
- Legal footer: All Rights Reserved © IQPREC
- Developer credit: Designed and Developed by Jacob Qumsiyeh — linked to jacobqum.dev

YOUR ROLE

- You are a senior full-stack engineer with 20+ years experience.
- Write pure HTML, CSS, and Vanilla JS — NO React, NO Vue, NO Angular, NO build tools.
- Every HTML page must work when opened directly in a browser or served by Express static files.
- Use CSS custom properties (variables) for all design tokens. No inline styles.
- Use ES6 modules with type="module" in script tags. Use fetch() for all API calls.
- Every page must have loading states, error states, empty states, and success states built in vanilla JS.
- NEVER recommend retired or inactive FPL players. validatePlayers() before every AI call.

PRICING — ONE PLAN

- Monthly: \$15/month. Season Pass: \$110/season (August to May 31st).
- 7-day free trial — no card required. After 7 days: pay or access blocked.
- One plan includes every single feature. No tiers. No gates.
- Every payment triggers official receipt email immediately.
- Trial countdown banner visible from day 1, color-coded by urgency.

IQPREC DESIGN SYSTEM

- Logo: precision targeting reticle SVG — three rings, four crosshairs, center dot. IQ in green, PREC in white.
- CSS variables in :root — --navy: #0B1929, --navy-card: #132235, --green: #00D97E, --gold: #F5A623, --red: #E8445A.

- Fonts loaded via Google Fonts link tag: Syne Bold (headings), DM Sans (body), Noto Sans Arabic (Arabic text), JetBrains Mono (data).
- Dark navy background always. Electric green sparingly. Skeleton loading states — never spinners.
- Mobile-first CSS: base styles for 375px, media queries for larger screens.
- Arabic RTL: add `dir="rtl"` to html element, use logical CSS properties (`margin-inline-start` not `margin-left`).

PENTAGON SECURITY — 8 LAYERS

- L1 — Input: Zod validation on every Express route before any code runs.
- L2 — Auth: JWT RS256. Access token 15 min. Refresh token 7 days httpOnly cookie.
- L3 — Rate Limits: Redis sliding window. Auth 5/15min/IP. AI 10/60sec/user.
- L4 — HTTP Headers: Full Helmet.js. CSP strict. HSTS preload. X-Frame-Options DENY.
- L5 — Data: Secrets in .env only. Stripe webhook signature always verified. Supabase RLS on all tables.
- L6 — Frontend: No tokens in localStorage. DOMPurify on all dynamic HTML.
- L7 — Infrastructure: SSH keys only on Contabo. UFW ports 80/443/2222 only. Fail2ban.
- L8 — AI: Block jailbreak patterns. FPL-only scope. 3 strikes warning, 5 strikes suspend.

ACTIVE PLAYERS ONLY

- ONLY recommend players active in the current FPL season from bootstrap-static API.
- After every sync: players NOT in API response get `status="removed"` in `fpl_players` table.
- `validatePlayers()` called before EVERY AI prompt. Throws `FPL_3010` if any player is removed.
- Mohamed Salah confirmed leaving Liverpool after 2025/26. Handled automatically by sync.
- Arab Stars Watch: `WHERE is_arab_player=true AND season=current AND status!="removed"`.
- NEVER hardcode player names. Always inject from live FPL API data.

AI EXPERT SYSTEM

- CAPTAIN RULES: FDR 1-2 preferred. Home +0.5 confidence. Form below 5.0 eliminates. Below 75% minutes eliminates. Above 65% rival ownership flags differential.
- TRANSFER RULES: Never recommend hit for less than 6 expected points. Wildcard signal at 4+ weak players.
- CHIP RULES: TC on confirmed DGW. BB with maximum bench. FH on blank GW. WC before DGW run.
- DIFFERENTIAL: Under 10% ownership + form above 6 + FDR 2 or below = strong pick.
- COMMUNITY DATA: Inject IQPREC Arab user ownership percentages into every captain and transfer prompt.

ARABIC AND RTL

- Arabic is DEFAULT. Website loads in Arabic. English toggle saves to localStorage.

- Language = Arabic: set `document.documentElement.dir = "rtl"` and `lang = "ar"`.
- Use CSS logical properties: `margin-inline-start`, `padding-inline-end`, `inset-inline-start`.
- All `fetch()` calls to AI endpoints send language parameter. Claude responds in Arabic when language is Arabic.
- Arabic FPL terms: Jawla (gameweek), Kaptin (captain), Inteqal (transfer), Tashkila (lineup).

DAILY SESSION START

- Start every Claude Code session with:
- "Read CLAUDE.md. Today is Day [X]: [Day Name]. We are building IQPREC at iqprec.com.
- Stack: HTML + CSS + Vanilla JS frontend, Node.js + Express backend.
- NO React. NO build tools. Pure HTML files.
- Standards: IQPREC design system, Pentagon security, mobile-first 375px, Arabic RTL.
- Zero shortcuts. Zero placeholders. Production-grade code. Confirm each step."



How to Run IQPREC — No Build Step Required

This is the beauty of HTML + JS. It just works.

Your Folder Structure After Day 1

iqprec/ ■■■■ CLAUDE.md ← Claude Code reads this ■■■■ server/ ← Node.js backend (the only npm install needed) ■■■■ src/ ■■■■ index.js ← Express server entry point ■■■■ routes/ ← All API routes ■■■■ services/ ← Business logic ■■■■ middleware/ ← Security, auth, validation ■■■■ package.json ■■■■ .env ■■■■ public/ ← ALL frontend files (no build needed) ■■■■ index.html ← Landing page ■■■■ dashboard.html ← Main dashboard ■■■■ captain.html ← Captain picker page ■■■■ lineup.html ← Lineup builder ■■■■ transfers.html ← Transfer advisor ■■■■ chat.html ← AI chat ■■■■ community.html ← Leaderboard ■■■■ competitions.html ← Competitions page ■■■■ billing.html ← Subscription page ■■■■ login.html ← Login ■■■■ register.html ← Register ■■■■ css/ ■■■■ design-system.css ← All CSS variables and base styles ■■■■ components.css ← Reusable component styles ■■■■ [page].css ← Page-specific styles ■■■■ js/ ■■■■ api.js ← All fetch() calls to backend ■■■■ auth.js ← Auth state management ■■■■ i18n.js ← Arabic/English translations ■■■■ [page].js ← Page-specific JavaScript

To Run the Project Every Day

1

Install server dependencies ONCE (first time only): `cd ~/vps/var/www/html/iqprec/server && npm install`. This is the ONLY npm install you ever need. No client npm install. No Vite. No build step.

✓ *node_modules exists in server folder. Takes 2-3 minutes max.*

2

Copy your .env file into the server folder: `cd server && cp ../.env.example .env` Then fill in your real values (Supabase URL, Stripe keys etc).

✓ *.env file exists in server/ with real values filled in.*

3

Start the server: `cd server && node src/index.js` Or with auto-restart: `node --watch src/index.js` The server serves BOTH the API AND the frontend HTML files via Express static file serving.

✓ *Terminal shows: IQPREC server running on port 5000*

4

Open your browser and go to: `http://localhost:5000` You see the IQPREC landing page immediately. To see changes: edit the HTML or CSS file, press F5 in browser. That is it. No compilation. No waiting.

✓ *Landing page visible in browser at localhost:5000*

5

To check changes in real time: open VS Code with "code ." Edit any HTML file in public/ folder. Switch to browser. Press F5. Change appears instantly.

✓ *F5 refresh shows your changes immediately.*

THE KEY INSIGHT: Express serves the public/ folder as static files. When a user visits `iqprec.com/dashboard.html`, Nginx sends them the HTML file directly. The browser downloads it and runs the JavaScript. The JavaScript calls your API at `api.iqprec.com` to get data. No React runtime. No virtual DOM. No hydration. Just HTML and JavaScript.

Project Structure + CSS Design System

Phase 1 · Foundation · Everything else builds on this.

1

Create the complete folder structure: `iqprec/server/src/routes`, `server/src/services`, `server/src/middleware`, `server/src/config`, `server/src/db`, `server/src/jobs`, `server/src/bot`. Create `iqprec/public/css`, `public/js`, `public/images`, `public/icons`. Create all HTML page files as empty files: `index.html`, `dashboard.html`, `captain.html`, `lineup.html`, `transfers.html`, `differentials.html`, `player-intel.html`, `chips.html`, `mini-league.html`, `chat.html`, `community.html`, `competitions.html`, `arab-stars.html`, `billing.html`, `login.html`, `register.html`, `verify-email.html`, `onboarding.html`, `about.html`, `privacy.html`, `terms.html`, `cookies.html`, `referrals.html`, `ask-me.html`.

✓ All folders and HTML files exist. Is `public/` shows all pages listed.

2

Create `server/package.json` with all backend dependencies: `express@5`, `zod`, `helmet`, `cors`, `compression`, `cookie-parser`, `dotenv`, `bcryptjs`, `jsonwebtoken`, `@supabase/supabase-js`, `ioredis`, `bullmq`, `node-cron`, `@anthropic-ai/sdk`, `stripe`, `resend`, `telegraf`. Type: `module` in `package.json` for ES modules. Scripts: `dev` with `node --watch src/index.js`, `start` with `node src/index.js`. Run: `cd server && npm install`.

✓ `npm install` completes in under 3 minutes. Zero errors.

3

Create `public/css/design-system.css` with all CSS custom properties in `:root`: `--navy #0B1929`, `--navy-card #132235`, `--navy-elevated #1A3255`, `--green #00D97E`, `--green-dark #006B38`, `--green-light #E8FFF4`, `--gold #F5A623`, `--gold-light #FFF8EE`, `--red #E8445A`, `--red-light #FFF1F3`, `--blue #3B8BEB`, `--blue-light #EEF6FF`, `--white #FFFFFF`, `--off-white #F8FAFB`, `--text-dark #0D1B2A`, `--text-mid #344861`, `--text-light #6B7F99`, `--muted #9AAABB`, `--border #DDE3EE`. Radius variables: `--r-sm 6px`, `--r-md 10px`, `--r-lg 16px`, `--r-xl 24px`. Base reset: `*`, `html` `body` `margin 0` `padding 0` `box-sizing border-box`. Body: `background var(--navy)`, `color var(--white)`, `font-family DM Sans`.

✓ CSS variables visible in browser DevTools computed styles on any element.

4

Add Google Fonts link to every HTML page head section: `Syne` (weights 700 and 800), `DM Sans` (weights 400 and 500), `Noto Sans Arabic` (weights 400 and 600), `JetBrains Mono` (weight 400). Create `public/css/components.css` with reusable component styles: `.btn-primary` (green background, navy text, 44px height), `.btn-secondary` (transparent with green border), `.card` (navy-card background, border 1px solid rgba 255 255 255 0.08, border-radius `var(--r-lg)`), `.skeleton` (animated shimmer gradient background). All touch targets minimum 44px height.

✓ Buttons render with correct IQPREC colors. Fonts load correctly.

5

Create the base HTML template that ALL pages use. Every page must include: DOCTYPE html, html lang="ar" dir="rtl" (Arabic default), head with charset UTF-8, viewport meta, title, Google Fonts link, link to design-system.css and components.css, link to page-specific CSS. Body structure: div.app-shell containing nav.sidebar (desktop) and main.content. Before closing body: script type="module" src="/js/auth.js" and page-specific JS. Create public/js/auth.js: manages accessToken in memory (NOT localStorage), checks authentication state, redirects to login if not authenticated.

✓ *Base HTML template renders correctly in browser. Auth.js loads without errors.*

6

Create public/js/api.js: all fetch() calls to the backend API. Base function: apiFetch(endpoint, options) that automatically attaches Authorization: Bearer token header from memory, handles 401 by calling POST /api/v1/auth/refresh, retries original request with new token, on refresh failure redirects to login. Also handles JSON parsing and returns { success, data, error, message }.

✓ *apiFetch function handles token refresh transparently.*

7

Create public/js/i18n.js: translation system in Vanilla JS. Define translations object with ar and en keys containing every string in the app. Function t(key) returns the string in current language. Function setLanguage(lang): sets window.currentLang, updates document.dir and document.lang, calls applyTranslations() which finds all elements with data-i18n attribute and sets their textContent to the translation. Save language preference to localStorage. Load on page init. Arabic FPL terms: Jawla (gameweek), Kaptin (captain), Inteqal (transfer), Tashkila (lineup), Al-Form (form), Waraqat Al-Badal (wildcard).

✓ *Language toggle switches all text to Arabic and applies RTL. Works on F5 refresh.*

8

Create server/src/index.js: Express 5 app with full Helmet.js security headers, CORS for FRONTEND_URL only, compression, express.json limit 10kb, express.static serving the public/ folder so all HTML files are accessible, global error handler. Create server/src/config/env.js: Zod validation of all environment variables on startup. Create .env.example in server/ documenting: PORT, NODE_ENV, FRONTEND_URL, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, JWT_PRIVATE_KEY, JWT_PUBLIC_KEY, REDIS_URL, ANTHROPIC_API_KEY, ANTHROPIC_MODEL, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_PRICE_MONTHLY, STRIPE_PRICE_SEASON, RESEND_API_KEY, RESEND_FROM, TELEGRAM_BOT_TOKEN, ADMIN_EMAIL, ADMIN_IP_WHITELIST.

✓ *Server starts on port 5000. Visiting localhost:5000 serves the public folder.*

DAY
2

HTML Templates + Reusable Components

Phase 1 · Foundation · Every page uses the same layout system.

1

Build the sidebar navigation HTML component in `public/js/components.js`: `renderSidebar()` returns HTML string with IQPREC reticle SVG logo, all navigation links with Lucide SVG icons inline (Dashboard, Lineup, Captain, Transfers, Differentials, Chips, Mini-League, Chat, Arab Stars, Community, Competitions, Billing, Ask Me), trial countdown badge, language toggle button. All nav items have `data-i18n` attribute for translation. Active link gets `class="active"` with green left border indicator. Inject into `div#sidebar` on every dashboard page via JavaScript.

✓ *Sidebar renders correctly. Active page highlighted. Language toggle visible.*

2

Build the IQPREC reticle logo as an inline SVG function: `createReticleSVG(size, color)` returns SVG string with three concentric circles (opacity 30%, 60%, 100%), four crosshair lines, center dot. `createLogoHTML(variant)` returns full logo: reticle SVG + wordmark IQ in green + PREC in white. Used in sidebar, landing page hero, and loading screens. The reticle SVG draws in with a CSS `stroke-dashoffset` animation on page load.

✓ *Reticle logo renders and animates correctly at all sizes.*

3

Create the bottom navigation for mobile in `components.js`: `renderBottomNav()` returns HTML with 5 items (Dashboard, Lineup, Captain, Chat, More). Fixed to bottom with height 56px. Safe area padding: `padding-bottom env(safe-area-inset-bottom)`. Active item shows green underline. More item opens a slide-up panel with remaining nav items. Hidden on desktop (display none above 768px). Visible only on mobile.

✓ *Bottom nav visible at 375px. Hidden at 1280px. Safe area padding correct on iPhone.*

4

Create `public/css/components.css` additions for all reusable UI patterns: `.trial-banner` (green/yellow/red variants based on days remaining), `.stats-card` (navy card with colored accent top border, number in JetBrains Mono), `.player-circle` (colored circle with player initials, used in pitch view), `.progress-ring` (SVG ring for confidence percentage), `.skeleton-text` (animated shimmer for text loading), `.skeleton-card` (full card shimmer), `.toast` (notification, top-right LTR and top-left RTL). All components work without any JavaScript framework.

✓ *All component styles visible in the browser CSS panel.*

5

Create `public/js/ui.js`: utility functions for showing/hiding elements, `showToast(message, type)` displays a toast notification for 4 seconds then removes it, `showSkeleton(elementId)` replaces content with skeleton, `hideSkeleton(elementId)` removes skeleton and shows content, `showError(elementId, message)` shows error state with retry button, `showEmpty(elementId, message, ctaText, ctaAction)` shows empty state with CTA. `formatNumber(n)` formats large numbers with commas. `formatDate(d, lang)` formats date in Arabic or English.

✓ *showToast works. Skeleton shows and hides correctly. Empty state renders.*

6

Create the trial countdown banner as a reusable component: `renderTrialBanner(daysRemaining)` checks if user is in trial. 5+ days: green banner "X days left in your free trial". 3-4 days: yellow banner "X days left — subscribe to keep access". 1-2 days: red banner "Last chance! Trial ends tomorrow". 0 days: replace dashboard with full upgrade page showing both pricing options. Banner has a Subscribe Now button that opens the subscription modal. Inject at top of main content area on all dashboard pages.

✓ *Banner shows correct color for each scenario. Subscribe button opens modal.*

7

Create the competition milestone bar component: function `renderMilestoneBar(currentUsers, nextMilestone)` builds an HTML element showing a progress bar with current user count filling toward next milestone. Shows current count and next milestone number as labels. Animated fill using CSS width transition. Fetch current count from GET `/api/v1/stats/milestone` endpoint on page load. Used on landing page and dashboard competitions widget.

✓ Milestone bar shows live user count from database.

8

Create the referral share component: function `renderReferralCard(referralCode, referralCount)` builds HTML with unique referral link display, copy to clipboard button, WhatsApp share button (uses `navigator.share` on mobile, `wa.me` URL on desktop), progress bar showing `referralCount` of 5 minimum for competition entry, competition entry status badge (Not Qualified / Qualified / Competition Entered). Add CSS animation on the progress bar fill.

✓ Copy button works. WhatsApp opens on mobile with referral message. Progress bar animated.

DAY
3

Landing Page — `public/index.html`

Phase 1 · Foundation · The first page Arab managers ever see.

1

Hero section in `index.html`: full viewport height section with `class="hero"`. Background: deep navy with CSS background-image creating a subtle grid pattern (use CSS `repeating-linear-gradient` for pitch line effect). Eight floating green dots using CSS custom properties and keyframe animations (`@keyframes` float with `translate Y` and opacity variation). IQPREC reticle SVG injected by JavaScript with `stroke-dashoffset` draw animation. H1 in Syne Bold: IQ in green span then PREC in white — 42px desktop, 28px mobile. Arabic paragraph headline below. Two buttons: `.btn-primary` "Start Free Trial" and `.btn-secondary` "See How It Works". Scroll indicator at bottom bouncing via CSS.

✓ Hero fills viewport. Reticle animates on load. Zero overflow at 375px mobile.

2

Social proof section: three counter elements with `class="counter"` and `data-target` attribute containing the final number. JavaScript `IntersectionObserver` triggers `countUp` animation when section scrolls into view. `countUp(element, target, duration)` increments from 0 to target over duration ms. Fetch real numbers from GET `/api/v1/stats` (public endpoint). Comparison cards below: left shows generic FPL tool (grey, English only), right shows IQPREC (navy, green, Arabic reasoning, community data).

✓ Counters animate on scroll. Numbers come from real API.

3

Competitions section HIGH on the page — before features: Section heading in Arabic and English. Live milestone bar component injected by JavaScript. Three prize cards with CSS grid: 1st \$500 gold border, 2nd \$300, 3rd \$200. Three numbered steps for how to enter. Current competition status badge. Link to `/competitions.html` for full rules.

✓ Competitions section visible above the fold on most screens.

4

Features section: 12 feature cards in CSS grid (3 columns desktop, 2 tablet, 1 mobile). IntersectionObserver adds class="visible" as each card enters viewport. CSS transition: opacity 0 to 1 and translateY 20px to 0 with staggered delay using nth-child selectors. Each card: colored icon div, Arabic h3, English p. Pricing section below: two cards side by side. Monthly \$15 and Season \$110. 7-day trial callout on both. CSS flex gap layout.

✓ Cards animate in on scroll. Pricing shows both options clearly.

5

Palestine community section: heading in Arabic. Inline SVG map outline of Palestine with a location pin marker on Beit Sahour. Top 3 community managers fetched from API and rendered as leaderboard rows. FPL join code in a copyable code element with click-to-copy JavaScript. FAQ accordion section below: 8 questions using HTML details and summary elements (no JavaScript needed for expand/collapse — native browser behavior). Style details summary with custom arrow indicator using CSS.

✓ Map SVG renders. FAQ expands and collapses natively. Join code copies on click.

6

Footer HTML: left side All Rights Reserved © IQPREC. Right side: Designed and Developed by Jacob Qumsiyeh as an anchor tag with href="https://jacobqum.dev" target="_blank". Middle: navigation links to all legal pages. Above footer: Ask Me section mentioning Jacob and Simon by name. All footer text has data-i18n attributes for Arabic translation. In RTL mode (Arabic): left and right swap correctly using CSS logical properties.

✓ Footer renders in both Arabic RTL and English LTR correctly.

7

OG meta tags in index.html head: og:title, og:description under 160 characters, og:image pointing to a 1200x630 branded image (create a simple one using Canvas API or prepare a static PNG with IQPREC reticle logo on navy background). Twitter card meta tags. Canonical URL meta. Arabic meta description. When shared on WhatsApp the preview must show the IQPREC branded image and the Arabic description.

✓ WhatsApp link preview shows IQPREC branded image and description.

8

Mobile test of index.html at 375px: Hero fills screen without overflow. H1 text readable at 28px. Two CTA buttons stack vertically and are full width. Counters section readable. Features in single column. Pricing cards stack vertically. Competition prizes visible. FAQ expands correctly. Footer readable. Test in Chrome DevTools device toolbar at iPhone SE preset.

✓ Landing page perfect at 375px. No horizontal scroll. All content accessible.

The backend is identical to the React version — same Node.js, same Express, same Supabase, same Pentagon security, same JWT. The only difference is the frontend auth pages are plain HTML + JS instead of React components. All the security rules from CLAUDE.md apply exactly as before.

**DAY
4**
Pentagon Security + Full Database Schema

Phase 2 — Security · Same as original plan — just HTML frontend.

1. Run all Supabase migrations. Create all 20 tables identical to the original plan: users (with subscription_status, trial_ends_at, referral_code, referred_by), refresh_tokens, fpl_players (with is_arab_player, season, status fields), fpl_fixtures, user_squads, ai_recommendations, ai_cache, subscriptions, community_leagues, community_standings, gameweek_results, competitions, competition_entries, referrals, user_tutorials, ask_me_messages, feedback_submissions, receipts, ultimate_milestone_tracker. All tables have RLS Enabled.
2. Row Level Security policies: users (own row only), refresh_tokens (own tokens), user_squads (own squads), ai_recommendations (own records), fpl_players (public SELECT), community_standings (public SELECT), competitions (public SELECT). Same policies as original plan.
3. Security middleware in server/src/middleware/: helmet.config.js (full CSP, HSTS, X-Frame-Options DENY, remove X-Powered-By), cors.config.js (FRONTEND_URL only, never wildcard), rateLimiter.js (Redis sliding window: auth 5/15min/IP, AI 10/60sec/user, global 100/60sec/IP).
4. validateBody middleware: Zod factory accepting a schema, validates req.body, returns 400 with field errors on failure. sanitizeInput: strips HTML tags, removes null bytes. token.service.js: generateAccessToken RS256 15 minutes, generateRefreshToken 64 bytes hex, storeRefreshToken SHA-256 hash, rotateRefreshToken, revokeAllUserTokens.
5. checkSubscription middleware: allow if status=trial AND trial_ends_at in future. Allow if status=active. Block 402 TRIAL_EXPIRED with upgradeUrl, monthlyPrice 15, seasonPrice 110. Applied to all /api/v1/app/* routes after verifyToken.

**DAY
5**
Auth Backend — Zero Vulnerabilities

Phase 2 — Security · Same as original plan — just HTML frontend.

1. POST /api/v1/auth/register: identical error for existing and new email (enumeration prevention). bcrypt 12 rounds. Generate referral_code. Set trial_ends_at 7 days from now. Set subscription_status to trial. Send verification email async. Zero tokens before email verified.
2. POST /api/v1/auth/verify-email: hash token lookup, set email_verified true, issue RS256 access token (15min) and refresh token (64 bytes), store hash, set httpOnly Secure SameSite=Strict cookie. Return access token and safe user object.
3. POST /api/v1/auth/login: rate limit first. Timing-safe comparison even for unknown email. Same generic error for wrong email or wrong password. 5 failures locks account 15 minutes and sends security email.
4. POST /api/v1/auth/refresh: httpOnly cookie ONLY. Rotate token. On failure: clear cookie, return 401. POST /api/v1/auth/logout: revoke token, clear cookie. GET /api/v1/auth/me: return safe user object including

trial_ends_at, referral_code, referral_count.

5. Receipt email system: every payment calls sendReceiptEmail. Professional invoice: IQPREC logo, receipt number, date, plan, amount, next billing. Store in receipts table. Arrives within 60 seconds of payment.

DAY
6

Auth Frontend — Pure HTML Pages

Phase 2 — Security · Same as original plan — just HTML frontend.

1. public/login.html: standalone HTML page with no sidebar. Centered card layout. Email input, password input with show/hide toggle button, submit button. CSS: full viewport height, flex center, card with shadow. JavaScript in public/js/login.js: handleSubmit function calls POST /api/v1/auth/login via apiFetch, stores access token in memory (window.accessToken), stores user object in sessionStorage (NOT localStorage), redirects to /dashboard.html. Identical error message for wrong email or wrong password.
2. public/register.html: fullName, email, password with 4-stage strength bar (CSS width transitions), confirm password, language toggle, optional ref query param auto-fills referral code. JavaScript in public/js/register.js: validates all fields, shows strength meter, calls POST /api/v1/auth/register, shows success message to check email.
3. Trial countdown banner: inject at top of every dashboard page HTML file. public/js/trial.js: on page load calls GET /api/v1/auth/me, calculates daysRemaining from trial_ends_at, calls renderTrialBanner(daysRemaining) from components.js. Banner subscribe button opens subscription modal.
4. public/onboarding.html: 3-step page (not a component overlay). Step 1: language selection — two large buttons, clicking sets language and goes to step 2. Step 2: FPL team ID input with validation, confirm or skip. Step 3: trial confirmation showing all features and subscribe option. CSS step indicator at top.
5. Subscription modal: created once in JavaScript, appended to body, hidden by default. Opens when any subscribe button is clicked. Shows two pricing cards: \$15/month and \$110/season. Payment method selector. Submit calls POST /api/v1/billing/checkout, redirects to Stripe Checkout URL.

FPL Data Engine — Active Players Only

Phase 3 — Data · Active players only. Receipts. Dashboard.

1. `cache.service.js`: Redis wrapper identical to original plan. `getOrSet` pattern. TTL constants: `BOOTSTRAP` 6h, `FIXTURES` 1h, `PLAYER_SUMMARY` 2h, `USER_TEAM` 30min, `AI_PLAYER_INTEL` 24h, `AI_DIFFERENTIALS` 12h. Never use `KEYS` command.
2. `fpl.service.js`: ACTIVE PLAYERS ONLY. After every bootstrap fetch, upsert all players to `fpl_players` with current season. Any player NOT in new bootstrap gets `status="removed"` automatically. Mohamed Salah handled automatically — if not in 2026/27 bootstrap he disappears. `validatePlayers()` throws `FPL_3010` for any removed player.
3. FPL sync cron every 6 hours. Thursday 8am Asia/Jerusalem: invalidate gameweek caches. Arab players detected and flagged from FPL API nationality data.
4. All FPL routes identical to original plan. `GET /api/v1/fpl/bootstrap`, `/players/search` (current season only), `/fixtures`, `/my-squad`, `/validate-team`.
5. `public/js/fpl.js`: all FPL-related fetch calls using `apiFetch`. `fetchMySquad()`, `fetchBootstrap()`, `searchPlayers(query)` with 300ms debounce. All functions return clean data objects for use by page-specific JS files.

Billing + Receipt System

Phase 3 — Data · Active players only. Receipts. Dashboard.

1. Stripe setup identical: monthly price \$15 recurring, season pass \$110 one-time. Launch coupon `IQPREC_LAUNCH` 30% off. Webhook handler with `constructEvent` first, idempotency check, `checkout.session.completed` handling monthly vs payment mode.
2. `activatePlan`: updates users table, sends receipt email immediately, calls `updateMilestoneTracker` which checks if new competition milestone is reached and auto-creates competition + sends announcement email.
3. `public/billing.html`: page showing current plan status. JavaScript in `public/js/billing.js`: fetches `GET /api/v1/billing/status`, renders plan card (trial X days, monthly next billing, season expires May 31). Receipt history table. Subscription modal for upgrading.
4. Season pass expiry cron: daily midnight UTC finds expired season pass users, sets status `trial_expired`, sends warm renewal email with their season statistics.
5. Live milestone counter: `GET /api/v1/stats/milestone` returns `current_users` and `next_milestone`. Used on landing page and dashboard. Fetched fresh on each page load.

Dashboard — `public/dashboard.html`

Phase 3 — Data · Active players only. Receipts. Dashboard.

1. `dashboard.html` HTML structure: `nav#sidebar` (rendered by `components.js`), `main.content` containing: `div.stats-row` (4 stat cards), `section.captain-hook`, `section.squad-formation`, `section.recent-recommendations`, `section.leaderboard`, `div.competitions-widget`, `div.trial-banner-container`.

2. `public/js/dashboard.js`: on `DOMContentLoaded`, verify auth (redirect to login if not), inject sidebar, inject trial banner, fetch and render all sections in parallel using `Promise.all([fetchMySquad(), fetchBootstrap(), fetchCaptainPick(), fetchLeaderboard(), fetchMilestone()])`.
3. Stats row: 4 cards with skeleton loading while fetching. Gameweek card with live deadline countdown using `setInterval` every second. Countdown turns red when under 7200 seconds (2 hours).
4. Captain pick section: if no recommendation yet shows "Generate this week's captain pick" button with green pulse animation. If recommendation exists renders player circle, name, confidence ring SVG, fixture badge, Arabic reasoning text, community context line.
5. Squad formation: SVG element with player circles positioned by formation. Each circle has title element for tooltip on hover. Bench row below a dashed line divider. Tap on mobile shows detail panel sliding up from bottom.
6. Competitions widget: fetch `GET /api/v1/auth/me` for `referral_count`. If less than 5 show progress bar and large share button. If 5+ and competition open show qualified badge and rank. If no competition open show milestone progress toward next.

KEY ADVANTAGE of HTML + JS for AI features: Streaming AI responses with SSE (Server-Sent Events) works PERFECTLY with vanilla JavaScript. `const evtSource = new EventSource("/api/v1/ai/chat")` is native browser API. No React state management needed. No `useEffect`. Just clean event listeners. This is actually simpler and faster than the React version.

DAY
10

AI Expert Engine — The Core

Phase 4 — AI · Vanilla JS SSE streaming. Active players only.

1. AI service backend identical to original plan: `callClaude` with rate limiting (10/60sec/user), `streamClaude` for SSE streaming, `getCachedOrGenerate` for caching, expert FPL rules file injected into every prompt, community data injection, `validatePlayers()` before every prompt build.
2. `public/js/ai.js`: client-side AI helper functions. `streamAIResponse(endpoint, params, onToken, onDone, onError)`: creates `EventSource` connection, listens for message events, calls `onToken` with each token, calls `onDone` when stream ends. Perfect for chat streaming.
3. `callAIEndpoint(endpoint, params)`: uses `apiFetch` for non-streaming AI calls (lineup, captain, transfer). Returns the full response. Shows skeleton while loading. Handles errors with retry button.
4. Expert FPL rules server file: `fpl-expert-rules.js` with captain rules, transfer rules, chip rules, differential signals — all identical to original plan. Injected into every Claude API call system prompt.
5. Community intelligence injection: before every captain/transfer call, query `user_squads` for IQPREC Arab ownership percentages. Inject community context into prompt. This data only exists in IQPREC.

DAY
11

public/lineup.html + public/captain.html

Phase 4 — AI · Vanilla JS SSE streaming. Active players only.

1. `lineup.html` structure: main content with two-panel layout (CSS grid, 1fr 1fr desktop, single column mobile). Left panel: SVG pitch with player circles. Right panel: AI recommendation output area with skeleton loading.
2. `public/js/lineup.js`: on load fetch my squad (`validatePlayers` first in backend), render SVG pitch, add Generate Lineup button. On button click: show skeleton on right panel, call `POST /api/v1/ai/lineup` via `apiFetch`, render response: formation badge, starting 11 list with Arabic reasoning per player, bench priority order.
3. SVG pitch in `lineup.html`: `viewBox 0 0 300 420` rectangle representing pitch. Player circles positioned using absolute percentages based on formation. Each circle is a element with club color fill and element for abbreviated name.
4. `captain.html`: three cards layout. Card 1 has gold border and "Top Pick" label. Each card: player jersey circle div (CSS), player name, confidence ring (SVG), fixture badge, Arabic reasoning paragraph, community context line "X% of IQPREC Arab managers agree."
5. WhatsApp share button: creates an offscreen canvas element, draws the captain card content using Canvas API 2D context, calls `canvas.toBlob()`, uses `navigator.share` on mobile or constructs `wa.me` URL on desktop.

1. transfers.html: squad panel on left showing all 15 players with red warning icons on weak players (low form, injured, low minutes). Transfer suggestion cards on right: OUT player red card, arrow icon, IN player green card, price difference badge, hit analysis section.
2. public/js/transfers.js: auto-detect weak players before calling Claude (status not available, form < 3 with FDR > 3, chance_of_playing < 50). validatePlayers() in backend before prompt. Render transfer cards with urgency badges: Urgent red, Recommended yellow, Optional green.
3. differentials.html: five gem cards in CSS grid. Card header with ownership pill (red, showing how rare the player is), form visualization (5 colored squares for last 5 GWs using inline CSS), fixture badge, AI reasoning paragraph.
4. public/js/differentials.js: fetch GET /api/v1/ai/differentials (12h shared cache for all users). Render five cards. Number 1 card gets animated green shimmer CSS animation. Sort buttons update active class and re-render cards in sorted order.
5. Community context on both pages fetched from user_squads aggregation: "Only 3% of IQPREC Arab managers have this player." Displayed as a small badge on each recommendation card.

1. player-intel.html: search input at top with debounced fetch (300ms). Results dropdown below input. On player select: render full player card with stats grid (form, xG, xA, minutes ring SVG, bonus points). Below stats: AI scouting report rendered as HTML from markdown-like text.
2. public/js/player-intel.js: search calls GET /api/v1/fpl/players/search?q=X (current season only). Dropdown built by injecting HTML. On select: fetch GET /api/v1/ai/player-intel/:id?language=ar (24h cache). Render verdict badge: Buy green, Hold yellow, Sell red.
3. arab-stars.html: displays Arab players WHERE is_arab_player=true AND season=current AND status!=removed. Each player card shows two progress bars: global FPL ownership (blue) and IQPREC Arab community ownership (green). Note at top: "Only players active in the current Premier League season."
4. Salah handling: if Salah is not in the 2026/27 FPL bootstrap after June 15, the sync marks him removed. arab-stars.html will simply not show him. No special code needed. The active players filter handles it automatically.
5. public/js/arab-stars.js: fetch GET /api/v1/fpl/arab-stars. For each player render card with dual ownership bars. Show "Our community trusts this player more than average" badge when IQPREC percentage exceeds global.

1. chips.html: chip inventory row at top showing 4 chip cards (TC, BB, FH, WC) with played/available state. Visual season timeline below: horizontal scrollable container with GW markers from current to 38. CSS scroll-snap-type x mandatory for snapping. DGW markers have gold background. BGW markers have subtle red. Chip recommendation badges pinned at specific GWs.

2. `public/js/chips.js`: fetch POST `/api/v1/ai/chips`. Render chip inventory row from chips used data. Build timeline dynamically by inserting GW marker divs. Insert chip badge at recommended GW. On badge click expand reasoning panel below timeline.
3. `mini-league.html`: league ID input form centered on page. Submit calls GET `/api/v1/ai/mini-league-spy`. Loading state: spy icon SVG with CSS rotation animation. League table with rank medals emoji, team names, GW points, rank change arrows.
4. `public/js/mini-league.js`: render league table from response. Highlight own row with green background. Ownership bar chart using plain HTML divs with width percentage CSS (no chart library needed). AI spy report paragraph below table.

DAY 15

public/chat.html + Voice Summary

Phase 4 — AI · Vanilla JS SSE streaming. Active players only.

1. `chat.html` structure: full viewport height. CSS flex column: messages area (flex 1, overflow-y auto), input bar fixed at bottom. Messages: user messages right-aligned green bubble, AI messages left-aligned navy card. Typing cursor: a blinking green span appended while streaming.
2. `public/js/chat.js`: uses `streamAIResponse` from `ai.js`. On submit: add user bubble to DOM, create empty AI bubble with blinking cursor, open `EventSource` to POST `/api/v1/ai/chat`, on each token append to AI bubble text, on done remove cursor. Scroll to bottom after each token using `scrollIntoView`.
3. Suggested prompts on empty state: four Arabic prompt buttons. On click: fills input and submits. Prompts: "Who should I captain this week?", "Is this transfer worth it?", "Should I wildcard now?", "Analyze my team weakness."
4. Voice summary section on dashboard: GET `/api/v1/ai/voice-summary` returns a 60-second Arabic script. Dashboard shows "Weekly Arabic Briefing" card with expand button. On expand shows the full Arabic text. All player mentions use current season active players.
5. Jailbreak protection in backend: before any chat message reaches Claude, check against blocked pattern list. Off-topic requests get Arabic response: "I am specialized in FPL only." Abuse tracking: 3 strikes warning email, 5 strikes suspend chat.

DAY 16

public/whatif.html + Feedback System

Phase 4 — AI · Vanilla JS SSE streaming. Active players only.

1. Post-GW BullMQ job: after each GW finalizes, calculate actual vs AI recommended points for all users. Store in `gameweek_results` table. This is the backend feedback loop proving IQPREC value.
2. `whatif.html`: season summary card with large animated number counter (CSS counter-up animation using CSS custom properties and `@keyframes`). Per-GW table: green rows where AI beat user, red rows where user beat AI. Always show both — honesty builds trust.
3. `public/js/whatif.js`: fetch GET `/api/v1/ai/whatif`. Calculate total potential points from response data. Animate counter from 0 to final value using `requestAnimationFrame`. Render GW table with correct row coloring.
4. `public/feedback.html`: form for submitting feature ideas. Category select, title input, description textarea. Submit calls POST `/api/v1/feedback`. Show submission history below form. Past winners section with implemented features and winner names.

5. Feedback competition admin: when admin selects winner, automatic email to all subscribers announcing the Community Champion and their implemented feature. Winner gets permanent badge on community leaderboard.

DAY
17**public/community.html + Share Cards**

Phase 5 — Community · Viral growth and human support.

1. community.html: filter tabs (Beit Sahour, Palestine, Jordan, All Arabs) as CSS pill buttons. Leaderboard table below. JavaScript renders rows: rank medal emoji for top 3, team name, GW points, total, rank change arrow. Own row gets class="own-row" with green background.
2. public/js/community.js: fetch GET /api/v1/community/leaderboard?region=. Tab click updates active tab class and re-fetches with region parameter. Own row detected by comparing user ID from auth to leaderboard user IDs.
3. Share card generator: public/js/share.js exports generateCaptainCard(player, confidence, fixture, communityPct) function. Creates offscreen canvas 1080x1080. Draws navy background, reticle logo, player circle, player name, confidence ring, fixture badge, community percentage, IQPREC watermark. Returns blob URL.
4. Share buttons: downloadCard(blob) creates temporary anchor tag and clicks it. shareWhatsApp(blob): if navigator.share available and can share files, use it. Otherwise construct wa.me URL with text only.
5. Referral link share: generateReferralMessage(code, lang) builds a complete Arabic WhatsApp message including the referral link iqprec.com/register?ref=CODE. Share via navigator.share or wa.me.

DAY
18**public/competitions.html — Full System**

Phase 5 — Community · Viral growth and human support.

1. competitions.html: current competition status card at top (name, status badge, deadline, prize pool, entry count). How to enter: three numbered steps with icons. Milestone progress bar component. Prize cards: \$500 first, \$300 second, \$200 third.
2. public/js/competitions.js: fetch GET /api/v1/competitions/current for open competition data. Fetch GET /api/v1/auth/me for referral_count. Render My Competition Status section: if referral_count < 5 show progress and share button. If qualified show rank among qualified entrants.
3. Qualified entrants leaderboard: top 10 sorted by FPL points with referral count shown. Transparency builds trust. Admin closes competition, selects winners, marks prize paid via Wise, sends winner announcement email.
4. Feedback competition section on same page: form with category dropdown (New Feature, Bug Report, Improvement, Competition Idea), title, description. Past winners section with name, their suggestion, and prize amount. Shows IQPREC listens to users.
5. Competition terms section: entry rules, eligibility (active subscribers or trial users), prize payment within 30 days via Wise, fraud disqualification policy. All in Arabic and English.

DAY
19**public/ask-me.html + Emails + Telegram**

Phase 5 — Community · Viral growth and human support.

1. ask-me.html: message thread list on left (desktop) or top (mobile). Thread detail on right or below. New message form at bottom. JavaScript fetches GET /api/v1/ask-me/messages, renders threads. Unread threads have bold styling and green dot indicator.
2. Admin Ask Me panel (in admin.html): all messages reverse chronological, unread first. Inline response textarea with responder name selector (Jacob or Simon). Send response calls POST /api/v1/ask-me/respond. Triggers notification email to user.
3. 9 email templates using Resend: welcome, verification, receipt (professional invoice with IQPREC reticle header), trial day 3 reminder, trial expired warm, competition open announcement, competition winner congratulations, Ask Me response notification, Thursday weekly digest.
4. Thursday digest cron 9am Amman: for each active subscriber, fetch cached captain pick, build personalized email in user.language, send via Resend. Process in batches of 50 with 200ms delays.
5. Telegram bot: /start, /captain (current season player with community context), /deadline (Amman timezone hours remaining), /injury [player] (current season only), /lineup (cached), /transfers (cached), /competition (user referral count and entry status).

DAY 20

Onboarding Tutorial — Vanilla JS Overlay

Phase 5 — Community · Viral growth and human support.

1. Tutorial system in public/js/tutorial.js: TUTORIAL_STEPS array with 6 steps. Each step: targetSelector (CSS selector), titleAr, titleEn, descriptionAr, descriptionEn, position.
2. startTutorial() function: creates a full-screen semi-transparent overlay div. For each step: find target element using querySelector, get its getBoundingClientRect(), draw a pulsing green ring around it using an absolutely positioned div with box-shadow glow animation, position a tooltip card near the element.
3. Tooltip card HTML: step number, Arabic and English title and description, Previous button, Next button, Skip button. All vanilla JS DOM manipulation. No framework needed.
4. Mobile adaptation: if window.innerWidth < 768, always position tooltip at bottom of screen as a slide-up panel (CSS transform translateY animation) regardless of where the target element is. Target element scrolled into view using scrollIntoView(behavior: smooth).
5. Tutorial completion: remove overlay, show success toast in Arabic "You're ready to use IQPREC!", save completion to database via PATCH /api/v1/tutorial/complete. Does not restart on subsequent logins. Replay option in settings page.

DAY 21

public/admin.html + Analytics

Phase 5 — Community · Viral growth and human support.

1. admin.html: protected page that checks user.is_admin on load, redirects non-admins. Set is_admin=true in Supabase SQL: UPDATE users SET is_admin = true WHERE email = 'jacob@email.com'.
2. Admin dashboard: 4 stat cards (total users, MRR in USD, AI cost today, competition status). Below: tabs for Users, Ask Me, Competitions, Analytics, Broadcast.
3. Users tab: searchable table with email, subscription_status, trial_ends_at, referral_count. Manual actions: suspend user, mark as admin.
4. Competitions tab: all competitions with status. Actions: View Qualified Entrants (sorted by FPL points), Select Winners, Mark Prize Paid (records payment date and method), Send Announcement. Winner selection

triggers automatic announcement email.

5. PostHog analytics snippet added to all HTML pages: track user_registered, fpl_team_connected, subscription_started with plan type, trial_converted, trial_expired_without_converting, competition_entered, referral_converted, tutorial_completed, ask_me_submitted. Never track passwords or card numbers.

DEPLOYMENT IS EASIER WITH HTML + JS: No build step means deployment is just copying files. Copy the public/ folder to your Contabo VPS. Nginx serves it as static files. The server/ folder runs on Node.js with PM2. certbot adds SSL. Done. No build pipelines. No environment variable injection issues. This is why the old web worked so well.

DAY
22

Legal Pages

Phase 6 — Polish and Launch · No build step. Just copy and run.

1. public/about.html: IQPREC story — Jacob and Simon from Beit Sahour, built from zero. IQ + PREC meaning. Both founders listed. Palestinian roots, global vision. Arabic RTL and English LTR versions.
2. public/privacy.html: data collected, third parties (Stripe, Anthropic, Supabase, Resend, Telegram, PostHog), user rights, contact privacy@iqprec.com. Arabic and English.
3. public/terms.html: subscription terms, 7-day refund window, competition rules, FPL disclaimer (not affiliated), AI disclaimer (entertainment and assistance not financial advice). Arabic and English.
4. public/cookies.html: session cookie always on, PostHog analytics optional. Cookie consent banner on first visit using localStorage flag.
5. Footer on all pages: All Rights Reserved © IQPREC. Designed and Developed by Jacob Qumsiyeh hyperlinked to jacobqum.dev target blank.

DAY
23

Full Landing Page Polish

Phase 6 — Polish and Launch · No build step. Just copy and run.

1. Landing page final polish: IQPREC reticle SVG draws in with stroke-dashoffset CSS animation on page load. Floating particle dots use CSS custom property --i for staggered delays.
2. Competitions section prominently positioned. Live milestone bar fetching real user count. All three prize cards with trophy emoji and gold/silver/bronze styling.
3. Performance: add loading="lazy" to any images below the fold. Preload Google Fonts link in head. Minify CSS files using simple regex replacements (Claude Code can do this inline).
4. OG meta tags: og:image must be a real 1200x630 PNG file in public/images/og-card.png. Create this as a simple HTML file, screenshot it, or generate via Canvas API and save.

DAY
24

Security Audit

Phase 6 — Polish and Launch · No build step. Just copy and run.

1. Input audit: every Express route handler has Zod validation. Oversized body returns 413. Null bytes stripped. SQL injection in search returns empty not error.
2. Auth audit: identical response timing for wrong email vs wrong password. 5 failures locks account. RS256 in JWT header confirmed. Rotated refresh token rejected.

3. Active player audit: test that a player ID marked removed throws FPL_3010 before AI call. Arab Stars Watch returns empty if no active Arab players.
4. Frontend security: no tokens in localStorage (check in DevTools Application tab). DOMPurify wraps all innerHTML assignments. Check all innerHTML usages in public/js/ files.
5. Infrastructure: SSH password login rejected. Port scan shows only 80/443/2222. SSL Labs A+. SecurityHeaders.com A+.

DAY
25

Performance Optimization

Phase 6 — Polish and Launch · No build step. Just copy and run.

1. Add cache headers to Nginx config for static files: CSS and JS files get Cache-Control max-age 86400 (1 day). HTML files get no-cache so updates show immediately.
2. Lazy load images below the fold: add loading="lazy" attribute. Preconnect to Google Fonts domain in head. Preconnect to Supabase domain.
3. validatePlayers() performance: cache the Set of current active player IDs in Redis alongside bootstrap data (6h TTL). One Redis GET per request instead of database query.
4. Page speed target: Time to First Byte under 200ms (server is fast), First Contentful Paint under 1 second (HTML is tiny), Largest Contentful Paint under 1.5 seconds.

DAY
26

Mobile Polish — 375px Every Page

Phase 6 — Polish and Launch · No build step. Just copy and run.

1. Test every HTML page at 375px in Chrome DevTools device toolbar (iPhone SE preset). Check: no horizontal scroll, all text readable, all buttons tappable at 44px minimum.
2. Chat page specific: input bar must not be hidden by mobile keyboard. Use CSS: position fixed, bottom env(safe-area-inset-bottom). Viewport height: height 100dvh not 100vh.
3. PWA manifest: create public/manifest.json with name IQPREC, short_name IQPREC, start_url /, display standalone, background_color #0B1929, theme_color #0B1929, icons array. Link in all HTML head sections. Add to Home Screen works on iOS and Android.
4. Test on real phone: navigate to http://YOUR_LAPTOP_IP:5000 from your phone (must be on same WiFi). Check all pages on the actual mobile browser not just Chrome DevTools simulation.

DAY
27

RTL and Accessibility

Phase 6 — Polish and Launch · No build step. Just copy and run.

1. RTL audit: switch to Arabic in language toggle. Sidebar appears on RIGHT. Navigation items align right. Form inputs text flows right. Toast notifications appear top-LEFT. All check with dir="rtl" on html element.
2. CSS logical properties audit: grep all CSS files for margin-left, margin-right, padding-left, padding-right, left, right positioning. Replace with logical equivalents: margin-inline-start, margin-inline-end, inset-inline-start, inset-inline-end.
3. ARIA: every icon-only button has aria-label in Arabic when language is Arabic. Navigation has role="navigation" and aria-label. Loading sections have aria-busy="true". Toast notifications have role="alert".

4. Keyboard navigation: Tab through the entire dashboard page using only keyboard. Every button, link, and input must be reachable and activatable.

DAY
28

Production Deployment — Contabo VPS

Phase 6 — Polish and Launch · No build step. Just copy and run.

1. SSH into Contabo VPS. Install: nginx, certbot, python3-certbot-nginx, ufw, fail2ban, nodejs-22, pm2. Create iqprec system user.
2. UFW: deny incoming, allow 22/80/443. Fail2ban SSH protection. Nginx config: HTTP to HTTPS redirect, proxy /api/ to localhost:5000, serve /var/www/iqprec/public/ as static files for all other routes.
3. Copy public/ folder to /var/www/iqprec/public/ on the VPS. Copy server/ to /var/www/iqprec/server/. Create .env in server/ from your Bitwarden stored secrets. Run npm install in server/.
4. PM2: create ecosystem.config.js in server/ with cluster mode max instances. Run pm2 start ecosystem.config.js. Run pm2 startup and pm2 save.
5. SSL: certbot --nginx -d iqprec.com -d www.iqprec.com -d api.iqprec.com. DNS on Namecheap: A record for root and www pointing to Contabo VPS IP. A record for api subdomain pointing to Contabo VPS IP.

DAY
29

Smoke Test + Critical Fixes

Phase 6 — Polish and Launch · No build step. Just copy and run.

1. Registration to dashboard: email verify, FPL team connects, tutorial starts, trial countdown visible. Navigate all pages — no 404 errors.
2. Auth: wrong email and wrong password show identical error. 5 failures locks account. Token refresh works transparently.
3. Subscribe: Stripe test card 4242 4242 4242 4242. Receipt email within 60 seconds. Dashboard shows Active.
4. AI: generate captain pick in Arabic. Verify Arabic FPL terminology. Verify current season player (not Salah if he left). Verify community percentage shows.
5. Competitions: register two accounts, second uses referral link from first. Count increments. At 5 referrals: competition_qualified=true. Zero P0 or P1 issues before Day 30.

DAY
30

Launch Day — Beit Sahour

Phase 6 — Polish and Launch · No build step. Just copy and run.

1. Pre-launch: Stripe live keys, production .env on Contabo, FPL sync ran (current 2026/27 season players in DB), milestone tracker seeded, SSL A+ on all domains, Better Uptime green.
2. WhatsApp to 200 contacts in Levantine Arabic: "Team! Finally launched IQPREC — the first Arabic AI website for FPL! 7-day free trial, no card. \$15/month or \$110 season. Cash prize competitions — refer 5 friends to win \$500! iqprec.com — share it!"
3. Instagram stories: screen recording of Arabic AI captain pick working, competitions milestone bar, community leaderboard. Location tag Beit Sahour. Save all to Highlights.
4. When 20th user registers: Competition 1 opens automatically. Announcement email to all. This is the moment the viral engine starts.

5. Monitor: Sentry every 30 minutes, PostHog registration funnel, Ask Me inbox. Jacob and Simon respond personally within 2 hours on launch day. Target: 80+ registered, 10+ paying, Competition 1 open.



Emergency Prompts for Claude Code

Copy and paste when something breaks. Fill in the brackets first.

General Bug Fix

IQPREC Bug Fix. Read CLAUDE.md. ERROR: [exact error from browser console or terminal] FILE: [which HTML, CSS, or JS file] SEVERITY: [critical / high / medium] WHAT USER SEES: [describe] Stack is HTML + CSS + Vanilla JS frontend, Node.js + Express backend. NO React. Fix root cause. Explain what was wrong and how fix prevents recurrence.

Retired Player in Output

IQPREC Active Player Violation. Read Active Players Only rule in CLAUDE.md. PLAYER: [name and FPL ID] FEATURE: [which page showed this player] Check: 1) Is player in fpl_players with season=current AND status!=removed? 2) Is validatePlayers() called before the AI prompt for this feature? 3) Did last FPL sync run correctly? Clear AI cache for this feature.

HTML/CSS Layout Issue

IQPREC Layout Fix. Read CLAUDE.md Design System section. PROBLEM: [describe what looks wrong] PAGE: [which HTML page] VIEWPORT: [desktop / mobile 375px / both] LANGUAGE: [Arabic RTL / English LTR / both] Stack is pure HTML + CSS. No React. No Tailwind. Use CSS custom properties from design-system.css. Mobile-first: fix 375px first. Use CSS logical properties for RTL. [paste relevant HTML and CSS]

Payment Debug

IQPREC Payment Debug. Read CLAUDE.md billing section. PROBLEM: [describe issue] PROVIDER: [Stripe or Paytabs] ERROR: [paste exact error or webhook log] Check: webhook signature, event idempotency, activatePlan function, receipt email, milestone counter, competition auto-open. Fix completely.

API Error

IQPREC API Error. Read CLAUDE.md Pentagon Security and stack info. ENDPOINT: [which API route] ERROR CODE: [HTTP status and error code] REQUEST: [what was sent] RESPONSE: [what was returned] Backend is Node.js + Express with Zod validation on every route. Check: Zod schema matches what frontend sends, auth middleware order, checkSubscription middleware, rate limit not exceeded. Fix completely.



How to Use — From Today to Launch

Clear steps. No confusion. August 22, 2026 is the target.

Delete the Old Project and Start Clean

IMPORTANT: The old project in `~/vps/var/www/html/iqprec` has React files and a broken setup. The cleanest move is to delete it and start fresh with the HTML + JS structure. You have not built anything significant yet — Day 1 was just the skeleton. Starting fresh takes 30 minutes and saves you days of frustration.

1

Delete the old project and create fresh: `cd ~/vps/var/www/html && rm -rf iqprec && mkdir iqprec && cd iqprec`. Your `CLAUDE.md` is already saved — copy it back: you have it in this chat.

✓ *Fresh empty iqprec folder ready.*

2

Create the server folder and install dependencies: `mkdir server && cd server && npm init -y`. Then install: `npm install express@5 zod helmet cors compression cookie-parser dotenv bcryptjs jsonwebtoken @supabase/supabase-js ioredis bullmq node-cron @anthropic-ai/sdk stripe resend telegraf`. This is the ONLY npm install in the entire project.

✓ *server/node_modules exists. Takes 3-5 minutes. No client npm install needed.*

3

Create the public folder: `cd .. && mkdir -p public/css public/js public/images`. Create `CLAUDE.md`: `nano CLAUDE.md` and paste the system prompt. Then run Claude Code: `claude`

✓ *CLAUDE.md in root. public/ and server/ folders exist. Claude Code starts.*

4

Day 1 prompt to type into Claude Code — paste this exactly: "Read `CLAUDE.md`. Today is Day 1: Project Structure + CSS Design System. Stack: HTML + CSS + Vanilla JS frontend, Node.js + Express backend. NO React. NO Vite. NO build tools. Pure HTML files in `public/` folder. Server folder only has npm. Create all HTML page files, CSS design system file, Express server `index.js`, and `env.example`. Zero shortcuts. Confirm each step."

✓ *Claude Code builds Day 1 structure. Everything works on F5 refresh.*

5

To check your work every day: `cd server && node --watch src/index.js`. Open browser: `http://localhost:5000`. Edit any HTML or CSS file. Press F5. Changes appear instantly. No compilation. No waiting. No npm errors.

✓ *Changes visible in browser in under 1 second.*

TIMELINE: Buy `iqprec.com` today (\$11.28 on Namecheap). Delete old project and start fresh today — takes 30 minutes. Build Day 1 tonight. The FPL 2026/27 season starts August 22, 2026. Transfer window opens June 15, 2026. You have 80 days. 30 to build. 50 to test, polish, and market. You are building the first Arabic AI FPL platform in the world. It starts tonight.

Built from zero. For our families. For our future.

IQPREC · iqprec.com · Intelligence. Precision. Every Gameweek.

Jacob Qumsiyeh & Simon Haddad · Co-Founders

All Rights Reserved © IQPREC · Designed and Developed by Jacob Qumsiyeh · jacobqum.dev